

类的成员变量

- [1. Const成员](#)
 - [1.1. const成员变量](#)
 - [1.2. Const成员函数](#)
- [2. 静态成员](#)
 - [2.1. 静态成员简介](#)
 - [2.2. 静态成员变量](#)
 - [2.3. 静态成员函数](#)
 - [2.4. 调用静态成员](#)

1. Const成员

1.1. const成员变量

1. 初始化放在构造函数的成员初始化表中进行：
 1. 常量在初始化的时候必须被给值，而不是赋值，所以不能写在构造函数内
 2. 所以我们通过初始化表的方式完成。
2. 每一个Const变量是指对于这个对象的生命周期内是不变的
3. static const:编译器内的常量，所有的对象都是一样的，最好在定义的地方进行初始化。

```
1 class A{
2     const int x; //常成员变量
3     public:
4         A(int c):x(c){}
5 }
```

1.2. Const成员函数

1. 可以是函数也可以是参数
2. 我们将不修改对象内变量的值的时候，将对应方法声明为const

```
1 class A{
2     int x,y;
3     public:
4         A(int x1, int y1);
5         void f();
6         void show() const; //前后要保证一致，const在后面
7 };
8 void A::f(){x = 1; y = 1;} //编译器怎么能发现不是const的？转化为防止变量被赋值，见下
9 void f(A * const this); //上面的函数相当于这个
10
11 void A::show() const
12 {cout << x << y;}
13 void show(const A* const this); //上面的函数相当于这个，第一个const表示指向对象常量，
    后一个const表示指针本身是常量
```

```

14
15 const A a(0,0); //常对象:这个对象是不可以修改的
16 a.f(); //错误, 常对象无法调用非常方法
17 a.show(); //正确

```

1. 声明为const的对象只能调用常成员对象函数
2. 如果是非const的对象, 则都可以进行调用
3. 是否const方法真的就不能修改对象里面的值了呢? 不是,const只是语法上避免了, 但是不是完全不可修改

```

1 class A{
2     int a;
3     int & indirect_int;
4     public:
5         A():indirect_int(*new int){ ... }
6         ~A() {
7             delete &indirect_int;
8         }
9         void f() const{
10             //只要不是直接修改变量的值就OK
11             //引用本身是不能修改的, 所以编译器认为没问题
12             indirect_int++; //只是指向的内容发生了变化
13         }
14 };
15 //用a来做初始化

```

1. 关键词mutable:表示成员可以再const中进行修改, 而不是用间接的方式来做。
2. 去掉const转换: (const_cast<A*>(this)->x 转换后可以修改原来的成员

2. 静态成员

1. 放在类的外部, 只能初始化一次。
2. 一个类共享变量

2.1. 静态成员简介

1. 类刻画了一组具有相同属性的对象
2. 对象是类的实例
3. 问题:为什么不声明成全局变量, 而是声明成类的成员。
 1. 如果把这些共享变量定义为全局变量, 却缺乏数据保护
 2. 名污染
4. struct和class在封装上大致类似
 1. struct默认访问public
 2. class默认访问private

2.2. 静态成员变量

1. 静态成员变量是类对象所共享
2. 唯一拷贝

3. 遵循类访问控制
4. 必须放置在类外

```
1 class A{
2     int x,y;
3     static int shared;
4 };
5 int A::shared=0;//静态成员的初始化放在类的外部，只能被赋值一次，所以不再头文件中定义，而是在实现中定义，避免重复。并且定义的时候不用再写static
6 A a,b;
```

2.3. 静态成员函数

1. 只能存取静态成员变量，调用静态成员函数
2. 遵循类访问控制：在类上直接访问只能是静态成员变量
3. 类也是一种对象，可以通过类直接调用静态方法

```
1 class A{
2     static int shared;
3     int x;
4     public:
5         static void f() {shared}
6         void q() { x,shared}
7 };
```

2.4. 调用静态成员

1. 通过对象使用: `A a;a.f();`
2. 通过类使用: `A::f();`
3. C++支持观点"类也是对象"
 1. smalltalk

```
1 class A{
2     static int obj_count;
3     public:
4         A(){obj_count++;} //追踪创建了多少个对象
5         ~A(){obj_count--;}
6         static int get_num_of_obj(); //查看已经创建了多少个对象
7 };
8 int A::obj_count=0;
9 int A::get_num_of_obj() { return obj_count; }
```

4. 原则:谁创建，谁归还。解决方法:自动归还
5. singleton:单件模式:通过静态成员函数来创建对象

```
1 class singleton{
2     protected://构造函数外部不可以使用
3     singleton(){}
4     singleton(const singleton &);
5     public:
```

```
6         static singleton *instance() {
7             return m_instance == NULL?
8                 m_instance = new singleton: m_instance;
9         }
10        static void destroy() { delete m_instance; m_instance = NULL; }
11    private:
12        static singleton *m_instance; //保存对象的指针也是static的
13    };
14    singleton *singleton::m_instance= NULL; //初始化
```